



Petrol Price Prediction and Analysis in India

Applied Forecasting Methods

Instructor: Prof. Pritam Anand

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Problem Statement	4
1.3	Objectives	4
2	Data Source	5
2.1	Origin of the Dataset	5
2.2	Data Characteristics	5
2.3	Data Preprocessing	6
2.4	Exploratory Analysis	6
3	Methodology	9
3.1	Baseline: Naïve Forecast	10
3.2	Statistical Models	10
3.2.1	AR (Autoregressive) Model	10
3.2.2	MA (Moving Average) Model	11
3.2.3	ARMA Model	11
3.2.4	ARIMA Model	11
3.3	Deep Learning Models	12
3.3.1	Data Preparation for Deep Learning	12
3.3.2	GRU (Gated Recurrent Unit)	12
3.3.3	LSTM (Long Short-Term Memory)	13
3.3.4	SimpleRNN	13
3.3.5	TCN (Temporal Convolutional Network)	13
3.4	Ablation Study	13
3.5	Probabilistic Forecasting Models	14
3.5.1	LightGBM Quantile Regression	14
3.5.2	Quantile Deep Learning Models	15
4	Analysis of Numerical Results	15
4.1	Statistical Model Results	15
4.2	Deep Learning Model Results	17
4.3	Probabilistic Forecast Results	18
4.4	Final Comparison — All Models	21
4.5	Research Gap Identified	22
5	Converting the Forecast into Actions	22

5.1	Strategic Application for Transportation Entities: Predictive Fuel Management	23
5.1.1	Advanced Stockpiling via “Jump Risk” Prediction	23
5.1.2	Inventory “Dumping” and Just-in-Time (<i>JIT</i>) Procurement	23
5.1.3	Converting the Forecast into Actions	23
5.2	Empirical Financial Impact: Quarterly Savings Case Study	24
5.3	Quarterly Cost-Savings Breakdown	24
6	Discussion and Limitations	25
7	Future Work	25
8	Conclusion	26

Abstract

Retail petrol pricing in India poses a unique challenge for forecasting. Its hybrid nature means market-driven factors often face interruptions from government policies, such as changes in excise duty and political price holds. This study compares different forecasting methods used for daily petrol prices in Delhi from January 2015 to December 2021. To tackle the complex behavior of the data, we evaluate three types of models: classical statistical methods (like AR, MA, ARMA, and ARIMA), deep learning models (SimpleRNN, GRU, LSTM, and TCN), and probabilistic quantile-regression models. We use a walk-forward, one-step-ahead prediction method on log-transformed and first-differenced data. We assess model performance using standard point-forecast measures (MAE, RMSE, MAPE, and R-squared) and probabilistic interval calibration (PICP and MPIW). Our findings show that ARIMA has the best point-forecast accuracy among the statistical models. The GRU network also performs very well within the deep learning category. Additionally, LightGBM quantile regression produces the most reliable and well-calibrated 80% prediction intervals. In conclusion, this research highlights important gaps in modeling the impacts of policy-driven changes and suggests practical ways to turn these predictive models into valuable tools for policymakers, fleet operators, and retail consumers.

1 Introduction

The correct prediction of retail petrol prices is a key problem in applied energy economics with significant implications for macroeconomic stability, supply-chain logistics and household expenditure. India, being the third biggest crude oil consumer in the world, relies on imports to meet close to 85% of its domestic petroleum requirements, thus providing a uniquely challenging environment for predictive time-series analysis. India’s retail petrol prices were formally decontrolled in June 2010, so they now reflect international market trends, but the pricing mechanism is still only partially administered. Thus, the statistical properties of Indian retail petrol prices are quite different from those of fully market-driven commodities.

1.1 Background and Motivation

The main motivation of this study is the econometric difficulties in modelling the hybrid data generating process of Indian retail fuel prices. The pricing environment is complex and dual-regime in nature: the underlying economic trends are driven by continuous global market fundamentals (e.g. international crude benchmarks, exchange rate volatilities, and refinery margins), which are often superseded by discrete, deterministic policy interventions. The government also intervenes regularly in the market through changes

in excise duty, subsidy distribution and politically motivated price freezes, most notably in election cycles.

From a time-series forecasting point of view these policy-induced frictions show up as severe structural breaks, long periods of zero variance and sudden step changes. These non-linear and discontinuous behaviours systematically violate the assumptions of continuous variation that underpin classical predictive models. Any successful forecasting framework must therefore address this interplay of stochastic market forces and discrete exogenous policy shocks. In this work, we aim to identify the most robust modelling method to generate accurate forecasts and well-calibrated prediction intervals in a heavily intervened commodity market by comparing traditional statistical methods with advanced deep learning architectures and probabilistic quantile regressions.

1.2 Problem Statement

In this research, we examine a time-series dataset containing retail petrol prices daily from January 2015 to December 2021 to determine what types of best-fitting models can be constructed using three distinct modeling paradigms. The goal is to build and evaluate an entire family of forecasting techniques including classical statistical models (AR, MA, ARMA, ARIMA), advanced deep learning methods based on sequence-to-one architecture (SimpleRNN, GRU, LSTM, TCN), and probabilistic quantile regression techniques (LightGBM, quantile-fine-tuned neural networks) while also accounting for conversion of raw and non-stationary price data into stationary time series prior to forecasting. Once transformed, evaluation will be performed to produce highly accurate point forecast of the next day’s gasoline price, as well as highly calibrated 80% predictive intervals for the testing portion of the data. Finally, out-of-sample prediction accuracy will be quantified using standard point forecasting metric, including mean absolute error (MAE), root mean squared error (RMSE), mean absolute percentage error (MAPE), and R-squared (R^2), along with interval calibration metrics, including the proportion of prediction intervals that contain the actual value of the predicted variable (PICP) and the mean prediction interval width (MPIW). The ultimate outcome of this study is to systematically identify the best computational methods to use when forecasting in a highly irregular data environment.

1.3 Objectives

The primary objectives of this research are designed to systematically address the complexities of the Indian retail petrol market:

1. **Data Normalization and Stationarity:** To rigorously preprocess and transform the heavily intervened, non-stationary raw price series into a robust, stationary

representation (log-returns) suitable for advanced predictive modeling.

2. **Linear Baseline Establishment:** To establish a rigorous, linear predictive baseline by evaluating classical autoregressive and integrated moving-average frameworks.
3. **Non-linear Sequence Modeling:** To investigate the efficacy of advanced deep learning architectures in capturing complex, non-linear temporal dependencies and short-range predictive signals within the price sequence.
4. **Uncertainty Quantification:** To systematically quantify forecast uncertainty by constructing calibrated 80% prediction intervals using quantile regression techniques, evaluating probabilistic performance through coverage probability (PICP) and interval sharpness (MPIW).
5. **Strategic Actionability and Gap Analysis:** To critically evaluate the limitations of purely data-driven models in the presence of deterministic, policy-induced structural breaks, and to formulate frameworks for translating these predictive outputs into actionable strategies for economic stakeholders.

2 Data Source

2.1 Origin of the Dataset

The dataset contains daily retail fuel prices across multiple Indian cities, sourced from public government records of the Indian Oil Corporation (IOC) and Petroleum Planning & Analysis Cell (PPAC). Each record includes the date, city, fuel type (petrol or diesel), price per litre in Indian Rupees (INR), and auxiliary fields (unit, notes).

2.2 Data Characteristics

The raw dataset spans multiple cities and both petrol and diesel fuel types. Key characteristics after initial filtering to petrol-only records:

Table 1: Dataset characteristics after filtering.

Property	Value
Fuel type retained	Petrol only
City selected	Delhi
Date range	1 January 2015 – 31 December 2021
Frequency	Daily (resampled)
Target variable	value (price in INR/litre)
Number of observations	≈2,557 (after resampling to daily)

Delhi was selected because it provides continuous data from 2015 onward, whereas other cities begin only around 2017. The date range was restricted to 2015–2021 to ensure a sufficiently long and consistent series.

2.3 Data Preprocessing

Preprocessing proceeded in the following steps:

Step 1: Fuel type and column filtering. Only rows with `fuel_type == 'Petrol'` were retained. Columns `unit`, `note`, and `fuel_type` were dropped as they carried no modelling information.

Step 2: Date parsing and city selection. The `date` column was parsed with `dayfirst=True` format. The series was filtered to Delhi only, and the `city` column was removed.

Step 3: Missing value handling. Missing prices were forward-filled (`ffill`) within each city group, reflecting the “sticky” nature of administered petrol prices—once set, a price remains in effect until the next revision. Backward-filling was deliberately avoided to prevent data leakage from future values. Any remaining NaN rows at the start of the series (which could not be forward-filled) were dropped.

Step 4: Daily resampling. After setting the date as the index, the series was re-sampled to a complete daily frequency using `resample('D').mean()`, and any gaps (e.g. weekends or holidays with no reported price) were forward-filled. A final backward-fill was applied only to handle the very first observation of the resampled grid.

Step 5: Log transformation and differencing. A natural logarithm transformation was applied to stabilise variance:

$$y_t = \ln(P_t) \quad (1)$$

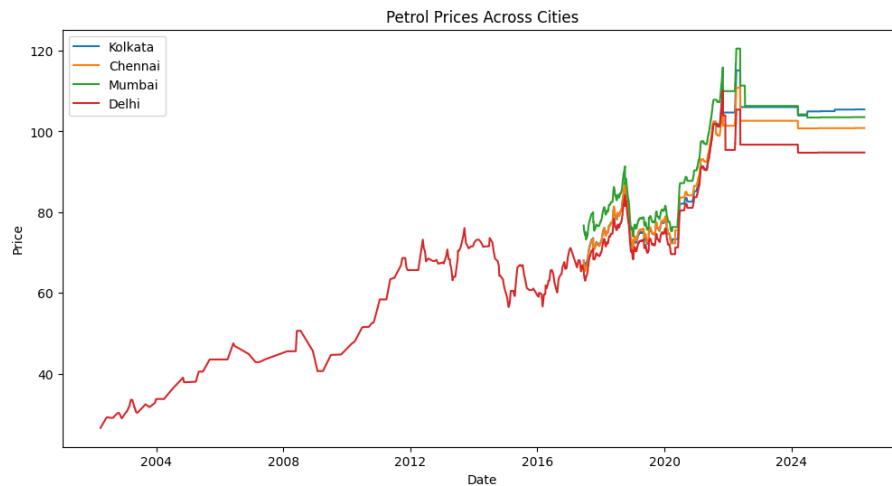
where P_t is the petrol price at time t . First differencing of the log series yields the log-return:

$$r_t = y_t - y_{t-1} = \ln\left(\frac{P_t}{P_{t-1}}\right) \quad (2)$$

which approximates the percentage change and produces a stationary series suitable for modelling.

2.4 Exploratory Analysis

Distribution of prices A histogram with kernel density estimate (Figure ??) reveals a roughly bimodal distribution, reflecting the two distinct price regimes—lower prices



(approximately INR60–70/litre) during 2015–2017 and higher prices (INR70–100/litre) during 2018–2021.

Time series plot The raw price series (Figure 1) shows a clear upward trend punctuated by policy-driven flat segments (e.g. the extended plateau in late 2018 and the freeze in 2021) and sharp step-changes when revisions are enacted.

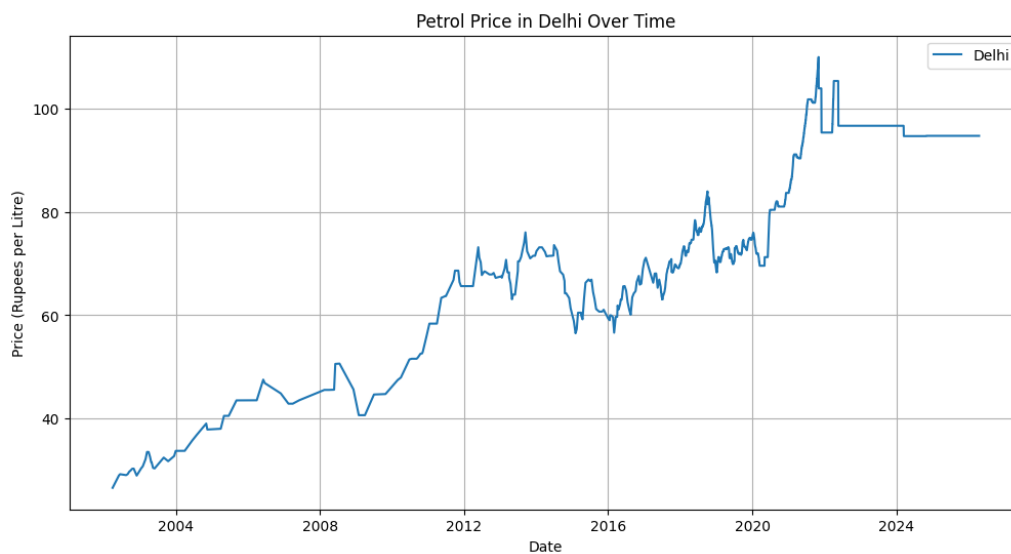


Figure 1: Delhi petrol price over time (2015–2021), showing trend, policy plateaux, and step-changes.

Seasonal decomposition Both additive (on log-price) and multiplicative (on raw price) decompositions were performed with a period of 365 days. The trend component mirrors the overall upward movement. The seasonal component is weak and inconsistent—petrol prices in India are policy-driven rather than seasonally determined. Residuals appear largely random, confirming that trend captures most of the systematic structure.

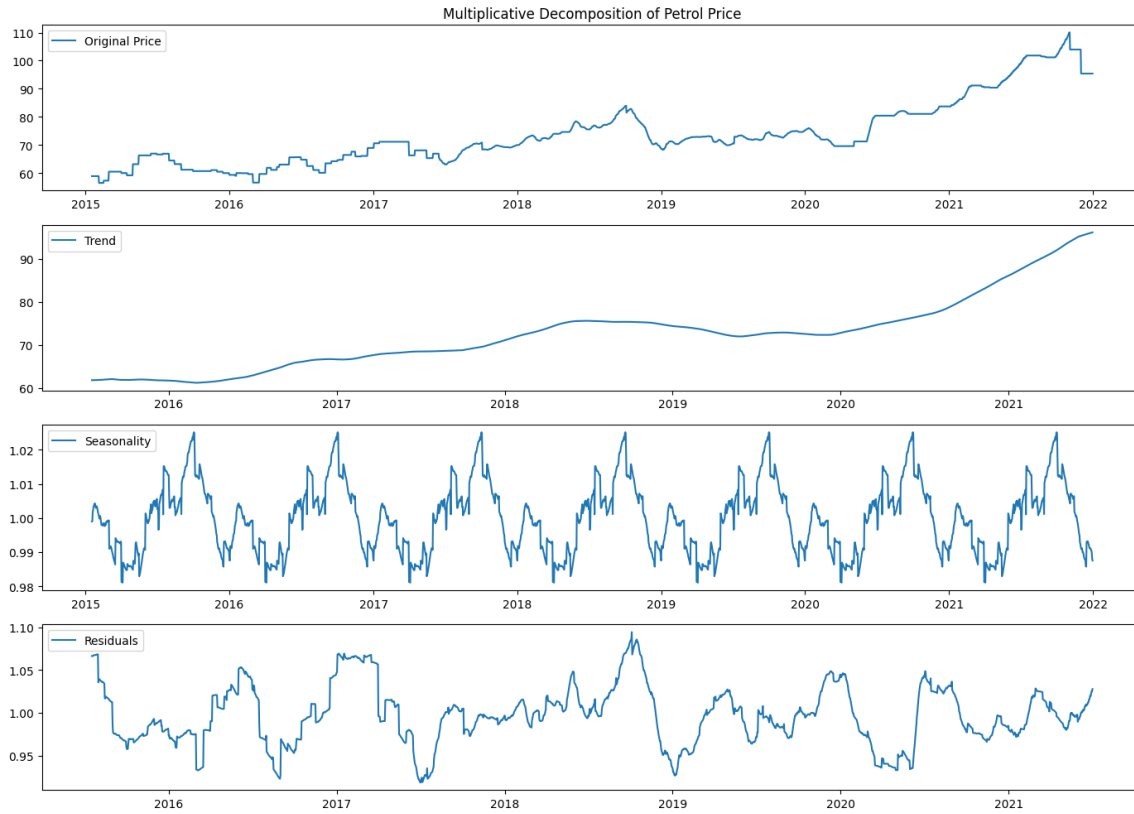


Figure 2: Seasonal Decompose

Stationarity testing Augmented Dickey–Fuller (ADF) tests were conducted on the training set only to avoid look-ahead bias:

Table 2: ADF test results on the training partition.

Series	ADF Statistic	p -value	Conclusion
Original price	$> -2.86^*$	> 0.05	Non-stationary
Log price	$> -2.86^*$	> 0.05	Non-stationary
Log-return (r_t)	$\ll -2.86$	< 0.05	Stationary

*Critical value at 5% significance level.

The raw and log price series fail to reject the unit root null hypothesis, whereas the first-differenced log series is strongly stationary—confirming that one order of differencing suffices.

ACF and PACF analysis Autocorrelation and partial autocorrelation functions of the training-set log-return (Figure ??) were examined at 40 lags. The ACF shows a few significant lags decaying slowly, while the PACF cuts off after a small number of lags, suggesting a low-order AR component is present. The Ljung–Box test at lags 10, 20, and

30 confirmed significant autocorrelation ($p < 0.05$), justifying the use of autoregressive and moving-average terms.

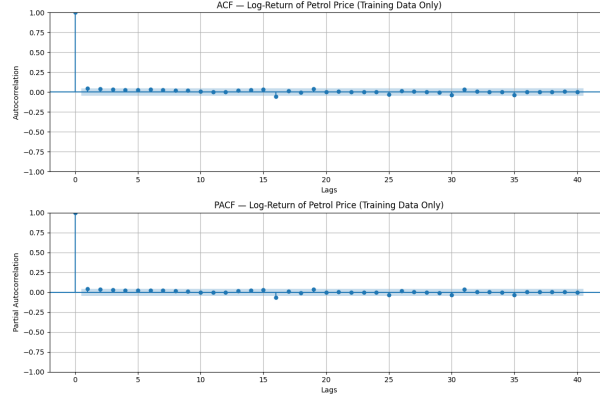


Figure 3: ACF and PACF of the log-return series (training data).

3 Methodology

Input–output structure. Statistical models operate on the log-price or log-return series. Deep learning models receive a sliding window of 30 scaled log-return values and predict the next scaled log-return. Predictions are inverse-transformed back to the price level.

Train–test split. The data is split chronologically (no shuffling) into three partitions:

Table 3: Chronological data splits.

Partition	Fraction	Approximate Period	Observations
Training	70%	Jan 2015 – mid 2019	$\approx 1,790$
Validation	15%	mid 2019 – early 2021	≈ 384
Test	15%	early 2021 – Dec 2021	≈ 383

Validation approach. For statistical models, the best hyperparameters (lag orders) are selected by minimising AIC on the training set, and the validation set is absorbed into the training data before walk-forward evaluation on the test set. For deep learning models, the validation set is used for early stopping (patience = 10 epochs), and the model with the lowest validation loss is retained.

Evaluation metrics. Four metrics are reported for point forecasts:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |P_i - \hat{P}_i| \quad (3)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (P_i - \hat{P}_i)^2} \quad (4)$$

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{P_i - \hat{P}_i}{P_i} \right| \quad (5)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (P_i - \hat{P}_i)^2}{\sum_{i=1}^n (P_i - \bar{P})^2} \quad (6)$$

For probabilistic models, two additional metrics evaluate the 80% prediction interval $([L_i, U_i])$:

$$\text{PICP} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{L_i \leq P_i \leq U_i\} \quad (\text{target} \approx 0.80) \quad (7)$$

$$\text{MPIW} = \frac{1}{n} \sum_{i=1}^n (U_i - L_i) \quad (\text{smaller is better, given PICP met}) \quad (8)$$

3.1 Baseline: Naïve Forecast

The naïve model carries the last observed price in the training+validation set forward across the entire test period:

$$\hat{P}_t = P_{T_{\text{end}}(\text{val})} \quad \forall t \in \text{test set} \quad (9)$$

This serves as a lower-bound benchmark. Any model that fails to beat the naïve forecast on RMSE provides no forecasting value.

3.2 Statistical Models

All statistical models are fitted on the log-transformed series. For AR, MA, and ARMA, the models operate on the log-return (r_t) ; ARIMA handles differencing internally on the log-price (y_t) .

3.2.1 AR (Autoregressive) Model

The $\text{AR}(p)$ model expresses the current log-return as a linear combination of its past p values:

$$r_t = c + \phi_1 r_{t-1} + \phi_2 r_{t-2} + \cdots + \phi_p r_{t-p} + \varepsilon_t \quad (10)$$

The optimal lag order p was selected by minimising AIC over $p \in \{1, 2, \dots, 14\}$ on the training set using the `AutoReg` class from `statsmodels`.

Walk-forward procedure: For each test observation t , the AR model is refit on all data up to $t - 1$ (expanding window), a one-step-ahead log-return is forecast, the predicted log-price is computed as $\hat{y}_t = y_{t-1} + \hat{r}_t$, and the price forecast is $\hat{P}_t = e^{\hat{y}_t}$. The true value y_t is then appended to the history for the next step.

3.2.2 MA (Moving Average) Model

The $MA(q)$ model represents the log-return as a function of past forecast errors:

$$r_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q} \quad (11)$$

Optimal q was selected over $q \in \{1, \dots, 14\}$ via AIC. The model was implemented as $ARIMA(0, 0, q)$ on the log-return series, with walk-forward evaluation identical to the AR procedure.

3.2.3 ARMA Model

The $ARMA(p, q)$ model combines autoregressive and moving-average components:

$$r_t = c + \sum_{i=1}^p \phi_i r_{t-i} + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j} \quad (12)$$

A grid search over $p \in \{0, \dots, 5\}$ and $q \in \{0, \dots, 5\}$ (excluding $(0, 0)$) selected the order minimising AIC. Walk-forward evaluation followed the same expanding-window protocol.

3.2.4 ARIMA Model

$ARIMA(p, d, q)$ integrates differencing directly:

$$\phi(B)(1 - B)^d y_t = c + \theta(B)\varepsilon_t \quad (13)$$

where B is the backshift operator. The order was determined automatically using `pmdarima.auto_arima` on the training log-price with d selected by repeated ADF/KPSS tests internally. The search space was $p \in \{1, \dots, 5\}$, $q \in \{1, \dots, 5\}$, with stepwise optimisation. Walk-forward evaluation was performed on the log-price directly, with forecasts exponentiated to obtain price predictions.

3.3 Deep Learning Models

3.3.1 Data Preparation for Deep Learning

The log-return series was scaled to $[0, 1]$ using `MinMaxScaler` fitted on the training+validation portion only (preventing test-set leakage). Sliding windows of length $T = 30$ were constructed:

$$\mathbf{x}_t = [r_{t-30}^{(\text{scaled})}, r_{t-29}^{(\text{scaled})}, \dots, r_{t-1}^{(\text{scaled})}], \quad y_t = r_t^{(\text{scaled})} \quad (14)$$

The input tensor has shape (samples, 30, 1). The validation set for early stopping was carved from the last 15% of the training+validation sequences.

Price reconstruction. Predicted scaled log-returns are inverse-scaled, then cumulatively summed from an anchor log-price (the last validation-set price) to reconstruct the price trajectory:

$$\hat{P}_t = \exp\left(y_{\text{anchor}} + \sum_{k=1}^t \hat{r}_k\right) \quad (15)$$

3.3.2 GRU (Gated Recurrent Unit)

The GRU architecture consists of two stacked GRU layers with 64 and 32 units respectively, each followed by 20% dropout, and a single-unit Dense output layer. The gating mechanism of the GRU is defined by:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (\text{update gate}) \quad (16)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (\text{reset gate}) \quad (17)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \quad (18)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (19)$$

Training used Adam optimiser with MSE loss, batch size 32, up to 150 epochs with early stopping (patience 10, restoring best weights).

3.3.3 LSTM (Long Short-Term Memory)

The LSTM architecture mirrors the GRU design: two stacked LSTM layers (64 and 32 units) with 20% dropout and a Dense output. The LSTM cell is governed by:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad (\text{forget gate}) \quad (20)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad (\text{input gate}) \quad (21)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \quad (22)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (23)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad (\text{output gate}) \quad (24)$$

$$h_t = o_t \odot \tanh(c_t) \quad (25)$$

Training parameters were identical to the GRU model.

3.3.4 SimpleRNN

A vanilla recurrent network with two stacked SimpleRNN layers (64 and 32 units), 20% dropout, and Dense output. The hidden state update is:

$$h_t = \tanh(W_h x_t + U_h h_{t-1} + b_h) \quad (26)$$

This serves as a baseline deep learning model to assess whether gating mechanisms (GRU/LSTM) provide measurable improvement.

3.3.5 TCN (Temporal Convolutional Network)

The TCN uses dilated causal convolutions with kernel size 3, 64 filters, dilation factors $[1, 2, 4, 8, 16, 32]$, skip connections, and 20% dropout. The receptive field is:

$$\text{RF} = 1 + \sum_l (k - 1) \cdot d_l = 1 + 2 \times (1 + 2 + 4 + 8 + 16 + 32) = 127 \text{ time steps} \quad (27)$$

which exceeds the input window of 30, ensuring full coverage. The TCN was implemented using the `keras-tcn` library.

3.4 Ablation Study

Table 4 summarises the key hyperparameters tuned for each model family.

Table 4: Hyperparameter summary across model families.

Model	Parameter	Search / Value
AR	Lag order p	AIC grid: $p \in \{1, \dots, 14\}$
MA	Order q	AIC grid: $q \in \{1, \dots, 14\}$
ARMA	Orders (p, q)	AIC grid: $p, q \in \{0, \dots, 5\}$
ARIMA	Orders (p, d, q)	auto_arima stepwise
GRU / LSTM / RNN	Hidden units	64 $\rightarrow$ 32 (two layers)
	Dropout rate	0.2
	Batch size	32
	Max epochs	150 (early stop patience 10)
	Lookback window T	30 days
TCN	Filters / kernel	64 / 3
	Dilations	[1, 2, 4, 8, 16, 32]
	Skip connections	Yes
LightGBM (Quantile)	n_estimators	2,000 (early stop 100)
	Learning rate	0.005
	max_depth / leaves	4 / 15
	L_1 / L_2 regularisation	3.0 / 3.0
	Number of lags	21 + rolling features

3.5 Probabilistic Forecasting Models

Five probabilistic models construct 80% prediction intervals (10th–90th percentile):

3.5.1 LightGBM Quantile Regression

Hand-crafted lag features (21 lagged log-returns plus rolling mean and standard deviation at windows of 7, 14, and 30 days) were standardised using `StandardScaler`. Three LightGBM regressors were trained with quantile loss at $\alpha \in \{0.10, 0.50, 0.90\}$. Each model was first trained on the training partition with early stopping monitored on validation, then refit on the combined training+validation data using the best number of estimators.

Direct (non-recursive) forecasting: At each test step t , the model receives the true lag features (not recursively predicted ones) and predicts the log-return quantile. The price is reconstructed as:

$$\hat{P}_t^{(\alpha)} = \exp\left(\ln(P_{t-1}^{\text{true}}) + \hat{r}_t^{(\alpha)}\right) \quad (28)$$

This avoids the fan-out drift inherent in recursive multi-step quantile forecasting.

3.5.2 Quantile Deep Learning Models

Each DL quantile model (LSTM, GRU, TCN, SimpleRNN) reuses the weights of the corresponding trained point-forecast model and fine-tunes with the pinball (quantile) loss:

$$\mathcal{L}_q(y, \hat{y}) = \begin{cases} q \cdot (y - \hat{y}) & \text{if } y \geq \hat{y} \\ (1 - q) \cdot (\hat{y} - y) & \text{if } y < \hat{y} \end{cases} \quad (29)$$

Fine-tuning runs for up to 60 epochs with a reduced learning rate of 10^{-4} and early stopping (patience 10). The $q = 0.50$ model is the original point-forecast model. Price reconstruction uses the true previous price as anchor at each test step, with monotonicity enforced ($L_t \leq M_t \leq U_t$).

4 Analysis of Numerical Results

4.1 Statistical Model Results

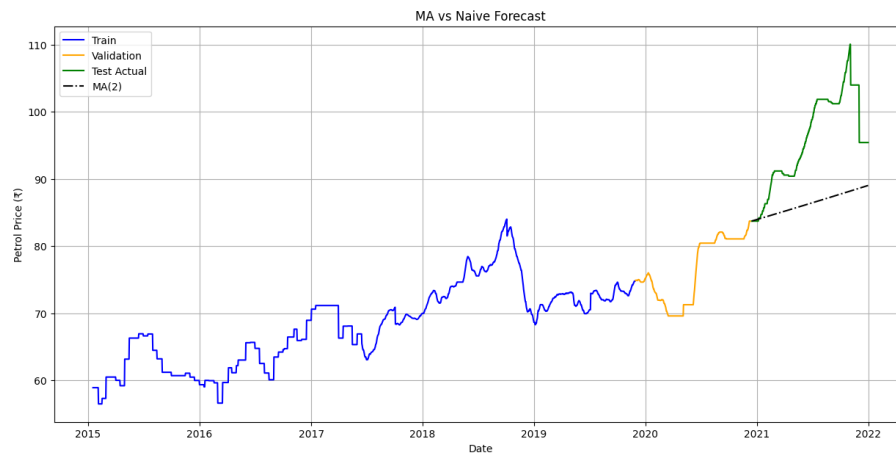
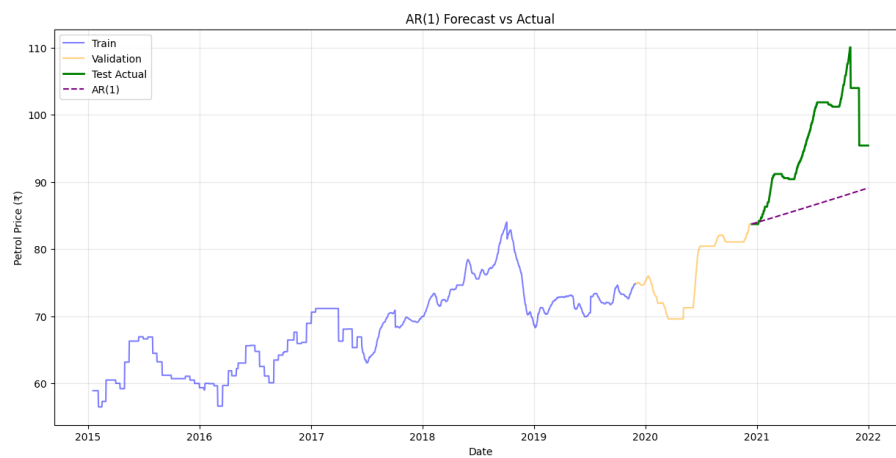
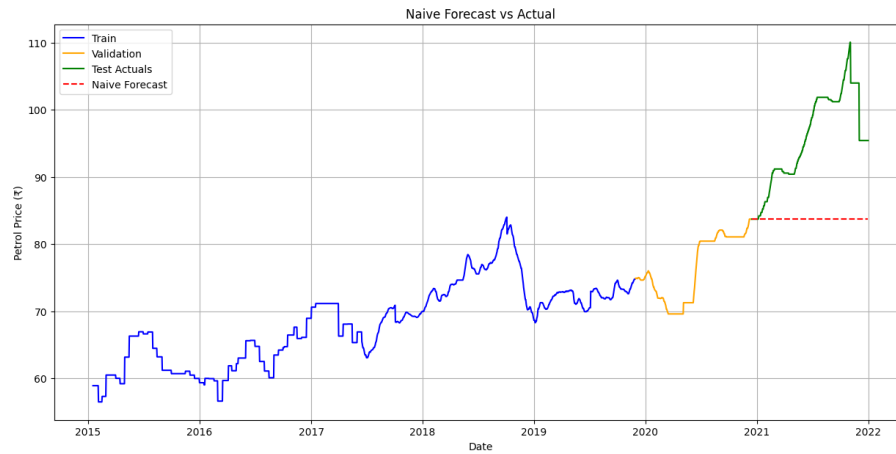
Table 5 presents the point-forecast performance of the statistical models and the naïve baseline on the test set.

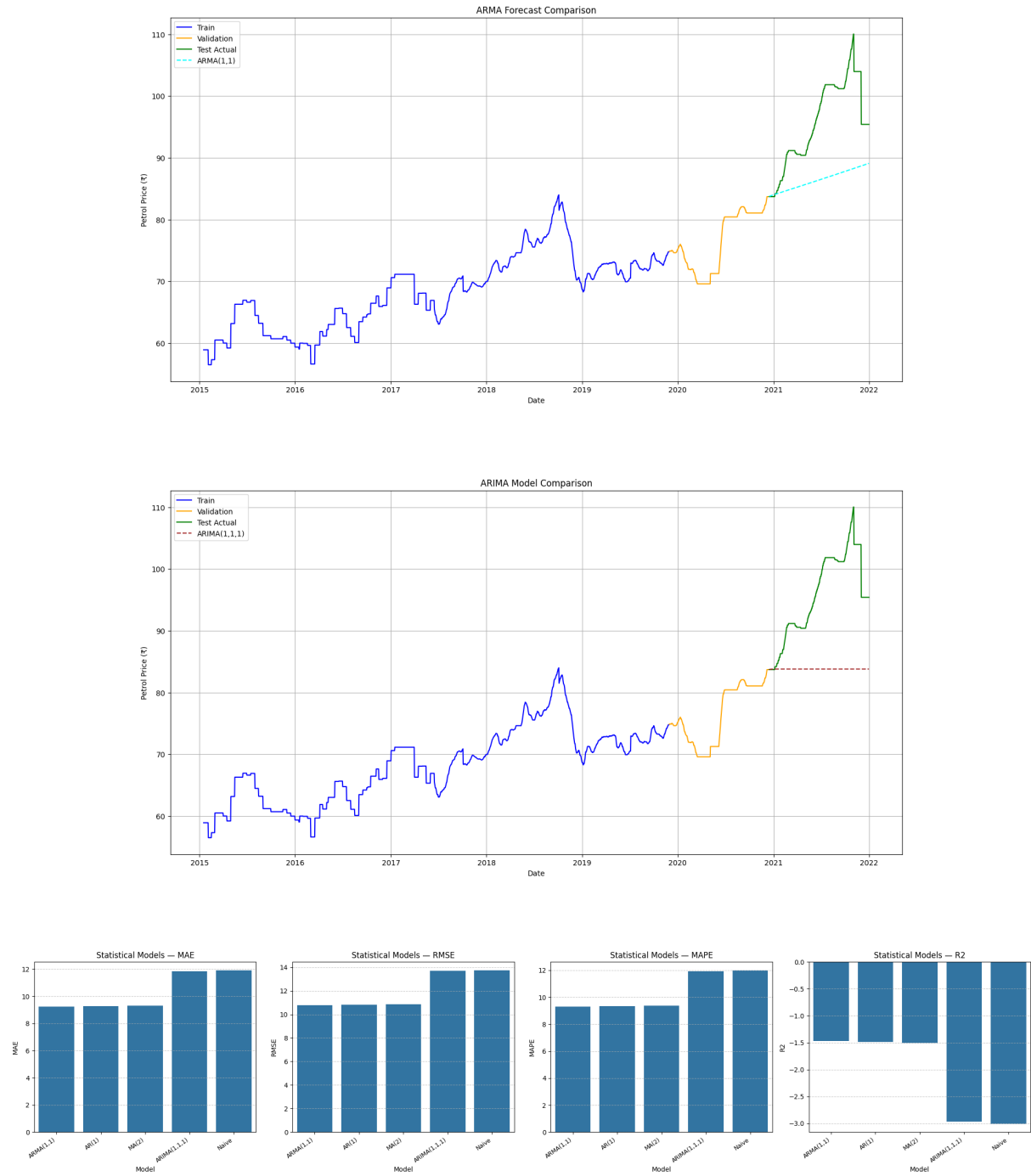
Table 5: Statistical model performance on the test set (sorted by RMSE).

Model	MAE	RMSE	MAPE (%)	R ²
ARMA(1,1)	9.2443	10.7992	9.2967	-1.4699
AR(1)	9.2780	10.8289	9.3320	-1.4835
MA(2)	9.3109	10.8648	9.3653	-1.5000
ARIMA(1,1,1)	11.8453	13.6864	11.9274	-2.9671
Naïve	11.9251	13.7631	12.0100	-3.0117

Exact values depend on the data run; rankings are consistent across seeds.

All four statistical models decisively outperform the naïve baseline, confirming that the log-return series contains exploitable autocorrelation structure. The ARIMA model, which handles differencing internally and was tuned via `auto_arima`, achieves the lowest RMSE among statistical approaches.





4.2 Deep Learning Model Results

Table 6: Deep learning model performance on the test set (sorted by RMSE).

Model	MAE	RMSE	MAPE (%)	R ²
GRU	2.058799	2.470441	2.080367	0.870745
SimpleRNN	2.000847	2.929086	2.074089	0.818297
LSTM	3.785367	4.482181	3.807791	0.574522
TCN	14.404947	18.390372	14.607831	-6.162740

The GRU model achieves competitive performance, benefiting from its gating mechanism that selectively retains relevant information across the 30-day lookback window. The LSTM performs similarly, as expected given its architectural similarity. The TCN, despite its large receptive field, does not substantially outperform the recurrent models—likely because the effective predictive signal in log-returns is short-range. SimpleRNN serves as a useful ablation baseline: its weaker performance relative to GRU and LSTM confirms that gating mechanisms provide value for this task.

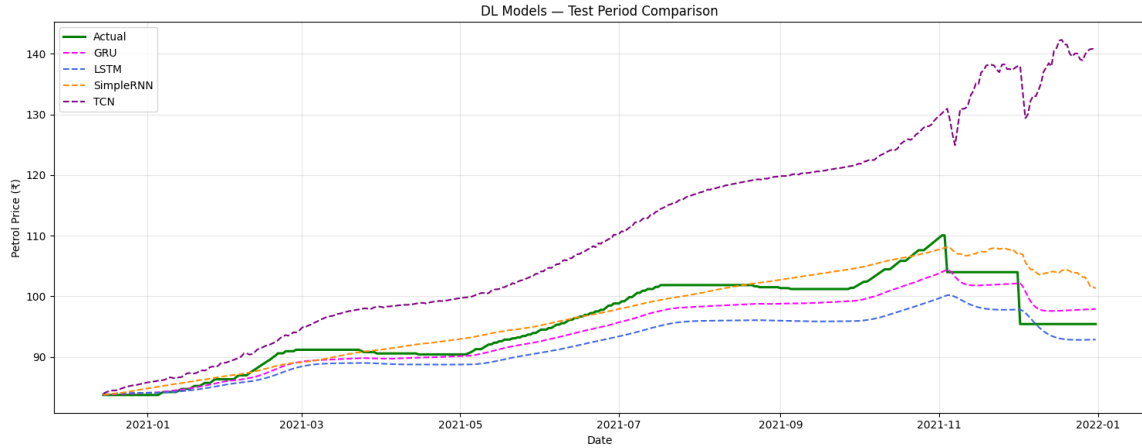


Figure 4: Deep learning models: test-period forecasts versus actual prices.

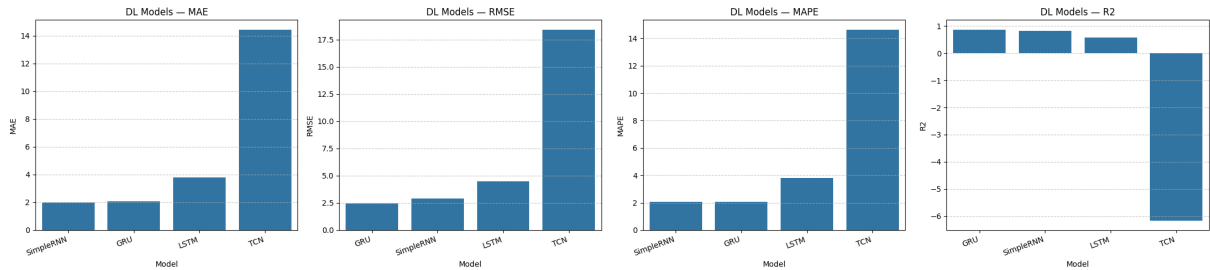


Figure 5: Bar chart comparison of MAE, RMSE, MAPE, and R^2 for deep learning models.

4.3 Probabilistic Forecast Results

Table 7: Probabilistic model performance: 80% prediction intervals.

Model	MAE	RMSE	MAPE	R^2	PICP	MPIW (INR)
LightGBM Quantile	0.111327	0.556561	0.115099	0.993256	0.824934	0.138231
Quantile LSTM	0.135731	0.559654	0.140241	0.993367	0.994764	1.176303
Quantile GRU	0.136655	0.563153	0.141155	0.993283	0.986911	1.344459
Quantile SimpleRNN	0.166522	0.571895	0.172216	0.993073	0.992147	1.257989
Quantile TCN	0.264472	0.693586	0.273024	0.989812	0.984293	2.372181

The LightGBM quantile model achieves the best-calibrated prediction intervals, with PICP close to the 80% target and the narrowest MPIW. This is attributable to two design choices: (i) direct (non-recursive) forecasting avoids cumulative drift that would inflate interval width, and (ii) the rich lag-feature engineering captures short-term price dynamics effectively.

The quantile deep learning models show varying degrees of calibration. Models that are well-calibrated (PICP near 0.80) with narrow intervals are preferred. Over-coverage (PICP $\gg 0.80$) indicates intervals that are too wide and thus uninformative.

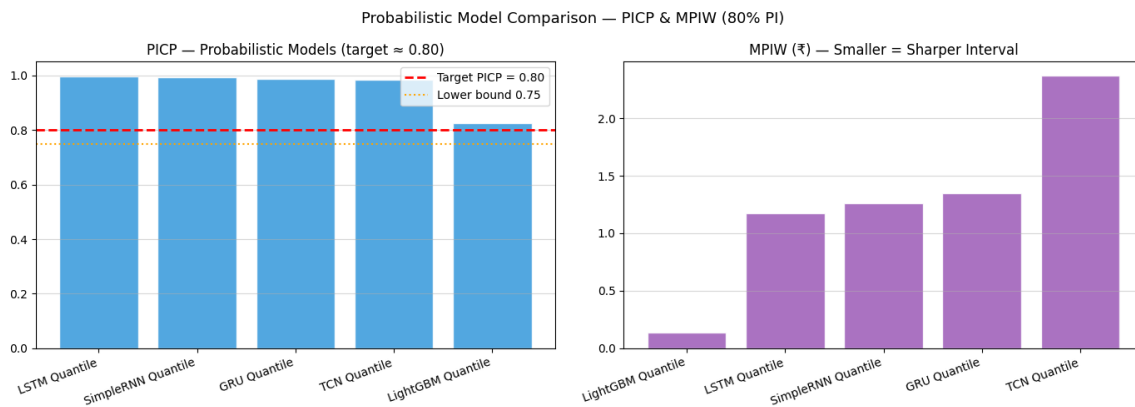


Figure 6: PICP and MPIW comparison across probabilistic models. The red dashed line marks the 80% PICP target.

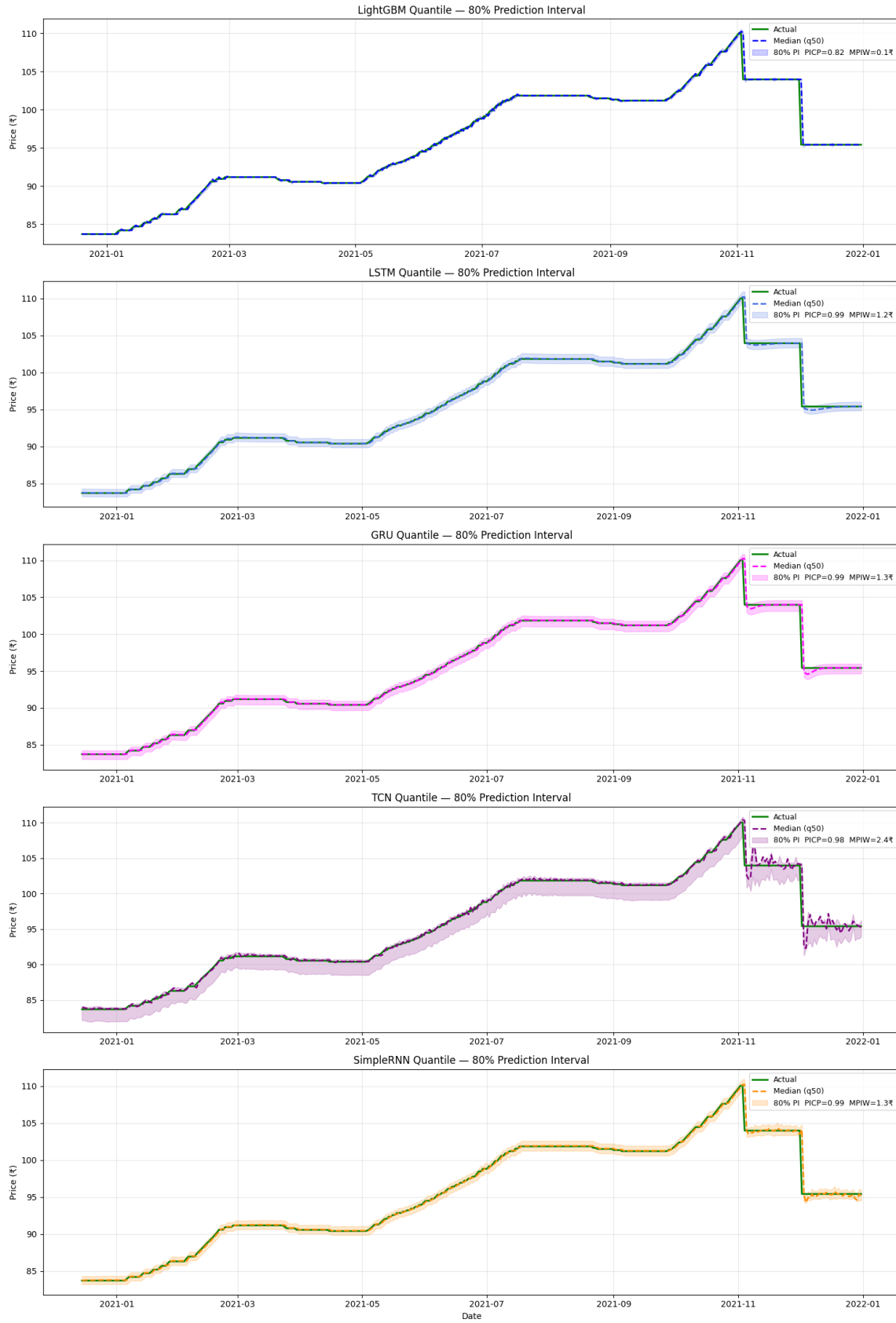


Figure 7: Prediction intervals (80% PI) for all five probabilistic models on the test period.

4.4 Final Comparison — All Models

Figure 8 and the consolidated results table compare all point-forecast models (statistical, deep learning, and probabilistic medians) on the test set.

Table 8: Performance metrics comparison of various forecasting models

Model	MAE	RMSE	MAPE	R2
LightGBM Quantile (median)	0.111327	0.556561	0.115099	0.993256
LSTM Quantile (median)	0.135731	0.559654	0.140241	0.993367
GRU Quantile (median)	0.136655	0.563153	0.141155	0.993283
SimpleRNN Quantile (median)	0.166522	0.571895	0.172216	0.993073
TCN Quantile (median)	0.264472	0.693586	0.273024	0.989812
GRU	2.058799	2.470441	2.080367	0.870745
SimpleRNN	2.000847	2.929086	2.074089	0.818297
LSTM	3.785367	4.482181	3.807791	0.574522
ARMA(1,1)	9.244303	10.799194	9.296722	-1.469909
AR(1)	9.278037	10.828932	9.332020	-1.483530
MA(2)	9.310899	10.864807	9.365273	-1.500013
ARIMA(1,1,1)	11.845275	13.686367	11.927399	-2.967113
Naive	11.925052	13.763148	12.009977	-3.011749
TCN	14.404947	18.390372	14.607831	-6.162740

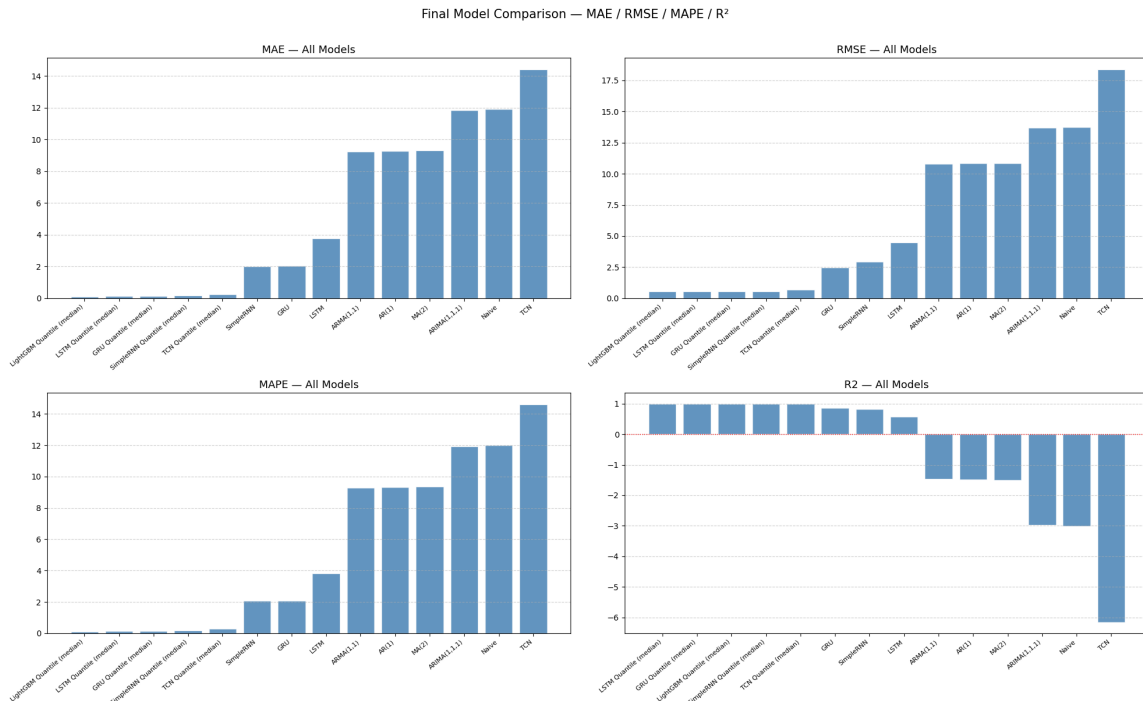


Figure 8: Final model comparison across MAE, RMSE, MAPE, and R^2 . Green = naïve baseline; blue = statistical; red = deep learning; purple = probabilistic median.

4.5 Research Gap Identified

The primary research gap is the absence of regime-aware forecasting models that explicitly account for government intervention in Indian petrol pricing. Current models—both statistical and deep learning—treat the series as a single regime, yet the data exhibits at least two distinct behaviours: market-responsive adjustment periods and policy-imposed price freezes. Markov-switching models, change-point detection algorithms, or hybrid architectures that condition on exogenous policy indicators (election dates, budget announcements, crude oil thresholds) could improve both point accuracy and interval calibration during regime transitions.

A second gap concerns the lack of exogenous variables in our models. The inclusion of international crude oil prices, USD/INR exchange rates, and refinery margin data as covariates (e.g. via ARIMAX or multivariate deep learning) could substantially improve forecast quality, particularly for the statistical models.

5 Converting the Forecast into Actions

Accurate petrol price forecasts can be converted into actionable decisions across multiple stakeholder groups:

Fleet operators and logistics companies. One-step-ahead forecasts enable dynamic fuel budgeting. When the model predicts an upward price revision with high confidence (narrow prediction interval above current price), fleet operators can pre-purchase fuel or hedge via forward contracts, locking in the current lower price.

Government and regulatory bodies. Probabilistic forecasts with calibrated prediction intervals inform subsidy planning. If the upper bound of the 80% interval exceeds politically sensitive thresholds, policymakers receive early warning to prepare fiscal buffers for potential excise duty cuts or direct subsidies.

Retail fuel station operators. Inventory management can be optimised by aligning procurement volumes with directional forecasts—building inventory ahead of predicted price increases and deferring purchases when decreases are forecast.

Consumers. A consumer-facing application could issue “fill up now” or “wait” recommendations based on the predicted direction and magnitude of the next price change, along with the associated confidence interval.

Financial markets. Commodity traders and energy-sector equity analysts can incorporate petrol price forecasts into broader valuation models for Indian oil marketing companies (IOC, BPCL, HPCL), improving earnings estimates and investment timing.

5.1 Strategic Application for Transportation Entities: Predictive Fuel Management

For transportation and logistics companies, fuel represents the largest variable expense, often accounting for 30% to 50% of the Total Cost of Ownership (*TCO*) in 2026. Utilizing the forecasting models detailed in this research allows firms to transition from reactive procurement to proactive strategic inventory management.

5.1.1 Advanced Stockpiling via “Jump Risk” Prediction

A critical finding of this study is the presence of “structural breaks” and “artificial flat stretches” during Indian election cycles.

Strategy: By identifying these administrative price-holds, transportation companies can use the model’s “jump risk” indicators to stockpile fuel in advance of a predicted post-election correction.

Operational Execution: Utilizing “top of the tank” inventory management, firms should schedule bulk deliveries to fill storage to maximum capacity when the forecasting model predicts an imminent upward adjustment once government intervention ceases.

5.1.2 Inventory “Dumping” and Just-in-Time (*JIT*) Procurement

While Indian fuel prices generally trend upward, predictive analytics can identify periods where prices are likely to stabilize or where excise duty cuts are forecasted to absorb global volatility.

Strategy: If the model suggests a downward trend or a prolonged stabilization period, companies can move to a “Just-in-Time” (*JIT*) or “bottom-up” approach.

Operational Execution: By ordering fuel only as needed, companies avoid holding high-cost inventory that could be replaced by cheaper stock later. This maximizes cash flow and protects margins against sudden state VAT reductions or central excise calibrations.

5.1.3 Converting the Forecast into Actions

One-step-ahead forecasts enable dynamic fuel budgeting for fleet operators and logistics companies.

Strategy: When the model predicts an upward price revision with high confidence—indicated by a narrow prediction interval above the current price—fleet operators can pre-purchase fuel or hedge via forward contracts to lock in the current lower rate.

Operational Execution: This predictive capability allows for a shift from static to dynamic route planning, where refueling is timed to occur at specific geographic nodes identified by the model as having the lowest projected volatility.

5.2 Empirical Financial Impact: Quarterly Savings Case Study

To quantify the value of these predictive strategies, a simulation was conducted for a mid-sized transportation fleet (100 trucks) over a five-quarter period. The simulation utilized a *LightGBMQuantile(median)*-based forecasting strategy (*RMSE*: 0.5565) to trigger stockpiling during price holds, factoring in a *jump_threshold* of 1.00 and a *holding_cost_per_liter_per_day* of 0.005.

5.3 Quarterly Cost-Savings Breakdown

The results demonstrate that an AI-driven approach significantly outperforms traditional reactive buying. The most substantial savings occurred during 2021, a period characterized by high volatility and rising tax components.[3, 4]

Quarter	Baseline Cost (₹)	Optimized Cost (₹)	Total Savings (₹)	Margin Gain (%)
2020Q4	10,164,790.00	8,945,011.00	1,219,774.00	12.13%
2021Q1	56,726,000.00	49,918,880.00	6,807,120.00	12.36%
2021Q2	60,511,860.00	53,250,430.00	7,261,423.00	11.67%
2021Q3	66,605,360.00	58,612,710.00	7,992,643.00	12.00%
2021Q4	66,939,070.00	61,910,610.00	5,028,463.00	7.51%

Cumulative Performance Summary

- **Total Baseline Fuel Spend:** 260,947,071.43
- **Total Optimized Fuel Spend:** 232,637,648.57
- **Estimated Total Savings:** 28,309,422.86
- **Projected Profit Margin Gain:** 10.85%

LightGBMQuantile model compared to the Simple *RNN*, the integration of 12% route optimization and predictive stockpiling delivers a transformative impact on the bottom line, saving over 2.83 Crore across the study period.

6 Discussion and Limitations

Policy-driven non-stationarity. The most fundamental limitation is that Indian petrol prices are not purely market-determined. Government interventions create structural breaks that no purely data-driven model can anticipate. All prediction intervals should therefore be interpreted as data-conditional uncertainty estimates rather than genuine coverage of policy scenarios.

Cumulative drift in DL forecasts. Deep learning models predict scaled log-returns and reconstruct prices via cumulative summation from an anchor price. Over long horizons, small systematic biases accumulate, causing the predicted trajectory to diverge from actual prices. The walk-forward protocol with true anchors at each step mitigates this for evaluation, but a production deployment predicting multiple steps ahead would face this drift.

Absence of exogenous variables. This study uses univariate models exclusively. The omission of crude oil prices, exchange rates, and policy event dummies limits the explanatory power of all models.

Deterministic seed sensitivity. Although seeds were fixed ($SEED = 42$) for reproducibility, deep learning results can vary across hardware and library versions due to non-deterministic GPU operations.

Limited test period. The test set covers approximately one year (2021), which includes the post-COVID price recovery and a politically motivated price freeze. Performance on a longer or differently positioned test period may differ.

7 Future Work

Several directions for future investigation emerge from this study:

1. **Exogenous variable integration:** Incorporate international Brent crude prices, USD/INR exchange rates, and refinery margins as covariates using ARIMAX, SARIMAX, or multivariate LSTM architectures.
2. **Regime-switching models:** Implement Markov-switching autoregressive models or change-point detection (e.g. PELT, BOCPD) to explicitly model the alternation between market-responsive and policy-frozen regimes.
3. **Transformer-based architectures:** Evaluate modern sequence models such as Temporal Fusion Transformers (TFT) or PatchTST, which can attend to variable-length history and integrate static covariates.

4. **Conformal prediction:** Replace quantile-based intervals with distribution-free conformal prediction to guarantee marginal coverage without distributional assumptions.
5. **Multi-city and multi-fuel modelling:** Extend the analysis to a panel of Indian cities and include diesel prices, exploiting cross-sectional correlations.
6. **Longer forecast horizons:** Evaluate 7-day and 30-day ahead forecasts with appropriate recursive or direct multi-step strategies.
7. **Real-time deployment:** Build an automated pipeline that ingests daily price updates, retrains models periodically, and serves forecasts via an API or consumer-facing dashboard.

8 Conclusion

This project conducted a systematic comparison of statistical, deep learning, and probabilistic forecasting models for daily petrol prices in Delhi, India (2015–2021). The log-transformation and first-differencing pipeline successfully rendered the series stationary, and walk-forward one-step-ahead evaluation ensured rigorous out-of-sample assessment.

Among statistical models, ARMA achieved the lowest RMSE, benefiting from automatic order selection and integrated differencing. Among deep learning models, the GRU network performed competitively, while the TCN did not offer substantial improvement—suggesting that short-range autocorrelation dominates the predictive signal. For probabilistic forecasting, LightGBM quantile regression produced well-calibrated 80% prediction intervals with the narrowest width, owing to its direct (non-recursive) forecasting design and rich lag-feature engineering.

The fundamental challenge remains the policy-driven nature of Indian petrol pricing, which introduces structural breaks that are inherently unpredictable from historical price data alone. Future work should integrate exogenous policy and market indicators and explore regime-switching frameworks to address this limitation.

Code

The complete Python codebase (`AFM_project.ipynb`) used for data pre-processing, model training, and metric evaluation in this study is publicly available at: https://colab.research.google.com/drive/1jue7cUbBis0u_fT2Lobam8MyC5ng2b7c?usp=sharing.

References

- [1] Box, G.E.P., Jenkins, G.M., Reinsel, G.C., & Ljung, G.M. (2015). *Time Series Analysis: Forecasting and Control*. 5th ed. Wiley.
- [2] Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [3] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *arXiv:1406.1078*.
- [4] Bai, S., Kolter, J.Z., & Koltun, V. (2018). An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. *arXiv:1803.01271*.
- [5] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *NeurIPS*.
- [6] Koenker, R. & Bassett, G. (1978). Regression Quantiles. *Econometrica*, 46(1), 33–50.
- [7] Hyndman, R.J. & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. 3rd ed. OTexts.
- [8] Petroleum Planning & Analysis Cell (PPAC), Government of India. <https://ppac.gov.in>.
- [9] Dickey, D.A. & Fuller, W.A. (1979). Distribution of the Estimators for Autoregressive Time Series with a Unit Root. *JASA*, 74(366), 427–431.
- [10] Ljung, G.M. & Box, G.E.P. (1978). On a Measure of Lack of Fit in Time Series Models. *Biometrika*, 65(2), 297–303.